Tutorial 01: Setting Up the Python Environment

In this Tutorial, we'll talk about:

- Installing Python
- Using Anaconda and virtual environments
- Managing packages with **pip** and **conda**
- Version Control with Git
- Organizing data science projects for real-world scalability

1. Installing Python

There are two common ways to install Python:

1.1 Option 1: Official Python Installation

1. Visit python.org/downloads
2. Download the latest version
3. During installation:
   - Check the box: **“Add Python to PATH”**
   - Choose **"Customize installation"** → enable *pip* and *IDLE*
4. Verify installation: python --version

1.2 Option 2: Install Anaconda (Recommended)

Anaconda is a Python distribution that includes:

- Python
- Jupyter Notebook
- Conda (package and environment manager)
- Hundreds of data science libraries (*NumPy, Pandas, etc.*)

Steps

1. Visit anaconda.com/products/distribution
2. Download the installer
3. Follow the setup instructions
4. Open the **Anaconda Navigator** or **Anaconda Prompt**

To verify installation

conda --version

python --version

2. IDEs for Python

An **IDE (Integrated Development Environment)** is a software application that provides developers with tools to write, test, and debug code efficiently.
For Python, several IDEs and code editors are widely used, each with strengths and weaknesses.

2.1. Popular IDEs and Editors for Python

- **IDLE**
  - Comes pre-installed with Python.
  - Simple and lightweight, good for beginners.
- **Jupyter Notebook**
  - Interactive environment for data science.
  - Ideal for data analysis, visualization, and machine learning experiments.
- **PyCharm**
  - Full-featured IDE (by JetBrains).
  - Great for large projects, debugging, and advanced features.
- **VS Code (Visual Studio Code)**
  - Lightweight, extensible, and popular.
  - Large ecosystem of extensions for Python, Git, and data science tools.
- **Spyder**
  - Designed for scientific computing.
  - Often used in combination with Anaconda.

Comparison of Python IDEs

IDE / Editor	Best For	Pros	Cons
IDLE	Beginners, quick scripts	Simple, comes with Python	Very limited features
Jupyter Notebook	Data science, ML, research	Interactive, great for visualization	Not ideal for large projects
PyCharm	Professional, large applications	Advanced debugging, refactoring tools	Heavy, paid version for Pro
VS Code	General purpose, extensible	Lightweight, huge extension marketplace	Needs extensions for features

IDE / Editor	Best For	Pros	Cons
Spyder	Scientific & numerical computing	Integrated with Anaconda, MATLAB-like	Less flexible than VS Code

Tip:

- If you're starting with **data science** → use **Jupyter Notebook**.
- For **general Python development** → **VS Code** is a great balance of features and performance.
- For **enterprise projects** → consider **PyCharm Professional**.

3. Structure of Project in data science

A Data Science project should follow a clear structure to ensure clarity, scalability, and easy collaboration. This organization makes the project easier to understand, maintain, and extend.

A typical layout separates raw data, notebooks, source code, and outputs:

- 🗁 **my_project/**
  - 🗁 **data/**
    - 🗁 raw/ → Original data (CSV, JSON, Excel, etc.)
    - 🗁 processed/ → Cleaned data ready for analysis
  - 🗁 **notebooks/** → Jupyter Notebooks for exploration
  - 🗁 **src/**
    - **init**.py → Makes it a package
    - utils.py → Utility functions
  - 🗁 **models/** → Saved models
  - 🗁 **reports/** → Visualizations and figures
  - 🗁 **tests/** → Unit tests
  - requirements.txt → Dependencies
  - environment.yml → Anaconda environment config
  - README.md → Documentation
  - .gitignore → Ignore unnecessary files

🗁 Folder Explanations

- **data/** → Keep your raw input data and any cleaned/processed versions.
- **notebooks/** → For interactive work, experiments, and analysis in Jupyter.
- **src/** → Your Python scripts (data preprocessing, modeling, utilities).
- **models/** → Store trained models, serialized files (e.g., .pkl, .h5) and checkpoints; useful for versioning models and reproducibility.
- **outputs/** (reports and/or tests) → Store model outputs, visualizations, or exported results.

- **requirements.txt** → Tracks the exact Python packages used (`pip freeze > requirements.txt`).
- **environment.yml** → Alternative for Conda environments (`conda env export > environment.yml`).
- **README.md** → Explains what the project is about, how to set it up, and how to use it.
- **.gitignore** → Specifies which files and directories should be ignored by Git (not tracked or committed).

4. Virtual Environments in Python

A **virtual environment** is an isolated Python environment that allows to install specific packages and dependencies **without affecting the global Python setup or other projects**.

4.1. Benefits of Using Virtual Environments

- Keeps projects isolated
- Prevents dependency clashes
- Ensures reproducibility
- Facilitates collaboration

4.2. Creating a Virtual Environment (using `venv`)

Steps:

1. Open terminal / command prompt
2. Navigate to the project folder: cd my_project
3. Create the environment:python -m venv env
4. Activate it: Windows: (.\env\Scripts\activate) or Mac/Linux: (source env/bin/activate) You should now see (env) in your terminal.
5. Install libraries like : (pip install pandas numpy matplotlib)
6. Deactivate when done: (deactivate)

4.3. Creating a Virtual Environment (using Conda (Recommended)

If you use Anaconda, Conda environments are more powerful:

Steps:

1. conda create --name ds_env python=3.11
2. conda activate ds_env
3. conda install pandas numpy matplotlib scikit-learn

4. List all environments: conda env list

5. Package Management in Data Science: pip vs. conda

When working in Data Science, we don't **only** rely on programming languages like Python or R.
We also use **package managers** to install libraries, manage dependencies, and create isolated environments.
The two most popular tools are **pip** and **conda.**

pip (Python Package Installer) :

```
Official package manager for **Python**.
```

Conda :

Conda is a cross-platform package and environment manager that supports multiple languages (Python, R, C, etc.) and is widely used in the Anaconda ecosystem for handling dependencies and reproducible environments.

5.1. pip vs. conda: Which to Use?

Feature	pip	conda
Language support	Python only	Python, R, C, etc.
Package source	PyPI	Anaconda repositories (conda-forge)
Dependency handling	May cause conflicts	More reliable dependency resolution
Environment management	No	Yes
Speed	Faster, but less robust	Slower, but more stable
Binary support	Limited	Full binary support

Best practice:

- Use conda inside Anaconda
- Use pip outside Anaconda or when conda doesn't support a package

5.2. Main Python Libraries for Data Science

- **NumPy** → Core library for numerical computing, arrays, and linear algebra.
- **Pandas** → Data manipulation and analysis using DataFrames and Series.

- **Matplotlib** → Basic plotting library for visualizations.
- **Seaborn** → Statistical data visualization built on top of Matplotlib.
- **Scikit-learn** → Machine learning library for classification, regression, clustering, and preprocessing.
- **SciPy** → Scientific computing with advanced math, optimization, and statistics.
- **Statsmodels** → Statistical modeling and hypothesis testing.
- **TensorFlow** → Deep learning framework developed by Google.
- **PyTorch** → Deep learning library developed by Facebook (Meta), widely used in research.
- **XGBoost** → Gradient boosting library for efficient and scalable machine learning.
- **LightGBM** → Fast, distributed, high-performance gradient boosting framework.

5.3. Installing and Managing Libraries

a. Installing Packages with pip

pip is Python's default package installer. It's simple and widely used.

- **Install multiple libraries**

pip install numpy pandas

- **Install a specific version**

pip install pandas==1.5.3

- **Upgrade a package**

pip install --upgrade matplotlib

- **Uninstall a package**

pip uninstall seaborn

- **List all installed packages**

pip list

- **Freeze the current environment for sharing**

pip freeze > requirements.txt

- **Recreate the environment from the file**

pip install -r requirements.txt

b. Installing with conda (Anaconda Users)

conda is the package manager that comes with Anaconda. It's especially useful when managing libraries that have C or Fortran dependencies (e.g., NumPy, SciPy).

- **Install multiple libraries**

conda install numpy pandas

- **Install from conda-forge channel**

conda install -c conda-forge seaborn

- **Export environment configuration**

conda env export > environment.yml

- **Recreate an environment from a file**

conda env create -f environment.yml (then activate the enviroment defined in the first line)

6. Version Control with Git

Git is a **distributed version control system (DVCS)** that allows :

- Track and manage changes to the code.
- Work on different versions (branches) without breaking the main project.
- Collaborate with teammates through shared repositories (e.g., GitHub).
- Roll back to earlier versions if something breaks.

6.1. Basic Git Workflow

- **Initialize a repository (start version control)**

git init

- **Check status**

git status

- **lists all the remotes**

git remote -v

- **Stage changes**

git add # add specific file

git add . # add all files

- **Commit changes**

git commit -m "Your message"

- **View history**

git log --oneline --graph --decorate

- **Branches**

git branch # create branch

git checkout # switch branch

git checkout -b # create + switch

- **Update from GitHub**

git pull --rebase origin main

- **Push to GitHub**

git push origin main

7. Scnenario to set Up a data science projet

- Structure your project folders cleanly
- Use a virtual environment
- Install necessary libraries
- Track everything with Git
- Work in Jupyter notebooks

MYProject: Step-by-Step Setup

Step 1: Create Your Project Folder

mkdir MYProject

cd MYProject

Step 2: Set Up a Virtual Environment

python -m venv env

.\env\Scripts\activate (source env/bin/activate # On Linux/Mac)

or Set Up a Virtual Environment (with conda)

conda create -n myenv python=3.10

conda activate myenv

Step 3: Install Your Libraries

pip install numpy pandas matplotlib seaborn scikit-learn jupyter

Step 4: Freeze Your Environment

Freezing the environment means capturing all the installed Python packages along with their exact versions in the current virtual environment.

- requirements.txt: Tracks pip-installed packages

pip freeze > requirements.txt

- environment.yml: For Conda-based projects

conda env export > environment.yml

Step 5: Create a Clean Folder Structure

```
MYProject/
│
├── data/           # Raw and processed data
├── notebooks/      # Jupyter notebooks
├── src/            # Python scripts (cleaning, modeling)
├── outputs/        # Plots, charts, reports
├── env/            # Virtual environment (excluded from Git)
├── requirements.txt  # Dependency file
└── README.md       # Project documentation
```

Step 6: Connect to GitHub to Ensure Version Control, Reproducibility, and Collaboration

- Initialize Git

git init

- Ignore unnecessary files

echo "env/" >> .gitignore

- Stage and commit your project files

git add .

git commit -m "Initial setup"

- Push to GitHub (after adding the remote)

git remote add origin https://github.com/username/MyProject.git

git branch -M main

git push -u origin main

8. Best Practices for Managing Environments

- Use **one environment per project**
- Track dependencies with `requirements.txt` or `environment.yml`
- Initialize **Git from the start** (exclude `env/` in `.gitignore`)
- Separate **raw data** from **processed data**
- Document setup steps in your **README.md**
- Regularly update dependencies (security & compatibility)
- Pin package versions for consistency
- Share the same environment file across the team
- Don’t commit large datasets to Git